AI Engineering and Claude Architect · Part V, Claude Code

Claude Code, Lesson 5

Claude Code in CI/CD

The review job had been green for a week. Claude had declined some requests, the script read that as "no findings," and nobody knew. In CI, a run that says nothing and a run that didn't happen have to look different.

Module	AI12 of 13
Lesson	5 of 5
Prerequisites	AI12 L1–L4, AI03 L3, AI09 L3
Exam relevance	**Core: TS 3.6, 4.6.** Also TS 4.1, 4.5
Reading time	~35 minutes

Contents

1. Claude Code in CI/CD
2. Module Revision

Lesson 5 of 5 · Claude Code

Claude Code in CI/CD

Key idea

Run non-interactively with `-p`, get structured output with `--output-format json` and `--json-schema`, review with an independent instance, and feed in earlier findings. Then write the wrapper carefully: it must tell "no findings" from "no review," and it must assume that the repository it reads can be hostile.

> *Before: CLAUDE.md, skills and hooks, explicit criteria, permissions. Teaches: `-p`, structured output, permissions in unattended runs, independent review, dedupe across commits, test generation, GitHub Actions, and CI security. Used by: AI13.*

The hook: green doesn't mean reviewed

The docs say: "If a run ends without valid structured output, for example because Claude declined the request, the response has no `structured_output` field. Before your script uses `structured_output`, check that the field is present."

I wrote that wrapper twice. **The JSON is hand-written in the shape the docs describe, and no model ran.** Real output from `CiFindings.java`:

```
run with no structured_output:
  naive script: exit 0, 0 findings (the job goes green)
  this script:  exit 2, no structured_output: review didn't happen
process failed: exit 2, run failed, exit 1: review didn't happen
```

"Zero findings" and "the review didn't run" have to be different outcomes. This is TS 5.3's access-failure-versus-empty-result problem again, in a pipeline.

Intuition: the night-shift inspector

A factory has a night inspector who works alone. You can't ask them questions, so their instructions need to be complete. Their report has to fit a form, as a machine reads it. They mustn't inspect their own work. And if the form doesn't arrive, that has to mean "no inspection," rather than "no problems."

Mechanism

Non-interactive runs

`-p` (or `--print`) runs one prompt and exits. Without it, a job waits for input and hangs. I tested the invented alternatives on this machine's CLI, version 2.1.318. These fail before any model call:

```
$ claude --batch -p x      error: unknown option '--batch'
$ claude --ci -p x         error: unknown option '--ci'
$ claude --headless -p x   error: unknown option '--headless'
```

`CLAUDE_HEADLESS` doesn't appear in the docs I read. Exit codes, per the docs: 0 on success and non-zero on failure.

Input. Stdin works, and is capped at 10 MB, per the docs:

```
gh pr diff "$PR" | claude -p "Review this diff against the criteria in CLAUDE.md"
```

Structured output

`--output-format`	Gives
`text`	Plain text, the default
`json`	JSON with `result`, `session_id` and metadata
`stream-json`	Newline-delimited events

With `--json-schema`, the data arrives in a `structured_output` field:

```
claude -p "Review the diff" \
  --output-format json \
  --json-schema '{"type":"object","properties":{"findings":{"type":"array"}},"required":["findings"]}'
```

An invalid schema stops the run before it starts. I confirmed that here too:

```
$ claude -p x --output-format json --json-schema 'not-json'
Error: --json-schema is not valid JSON: JSON Parse error: Unexpected identifier "not"
```

Schema gives you shape, and it doesn't check whether a finding is true (TS 4.3).

Project context in CI

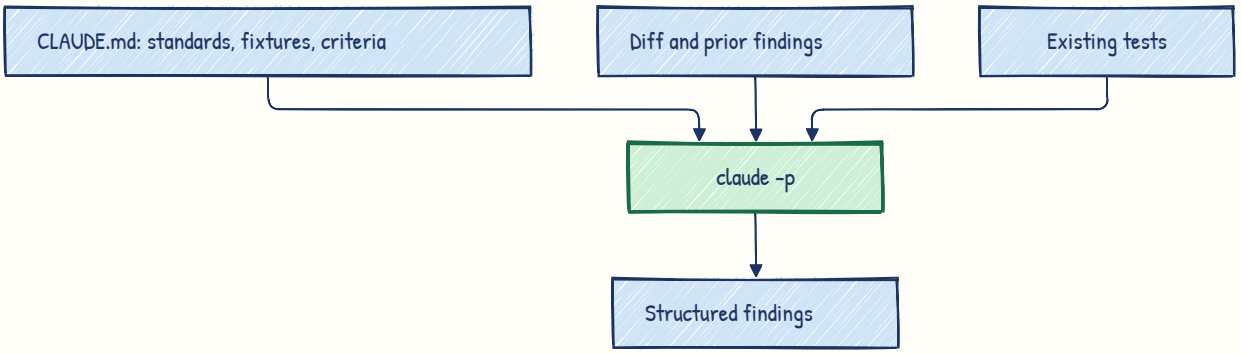


Figure 1. Everything the run knows has to be in the repository or the prompt. Nobody's there to answer.

Per the guide, CLAUDE.md gives CI its project context: testing standards, fixture conventions, and review criteria.

Exam answer and what to build today

The guide's answer is that **CLAUDE.md provides context to CI-invoked Claude Code.** That's how a plain `claude -p` behaves. The current docs also recommend `--bare` for scripted calls, and say it will become the default for `-p` in a future release. It skips auto-discovery of hooks, skills, MCP servers and CLAUDE.md, so that results are the same on each machine. With `--bare`, pass context explicitly with `--append-system-prompt-file`, `--settings`, `--mcp-config` and so on. It also needs `ANTHROPIC_API_KEY`, or an `apiKeyHelper`. Check this when you upgrade.

Permissions when nobody's there

From the docs: a Manual-mode `claude -p` run denies anything needing approval, unless something approves it. So it won't hang on a prompt, and it may quietly be unable to act.

Option	Effect
`--allowedTools "Bash(git diff *),Read"`	Pre-approves these
`--permission-mode dontAsk`	Denies anything that would prompt. For locked-down CI
`--permission-prompts none`	Tells Claude nobody can approve, so it shouldn't retry
`--max-turns`, `--max-budget-usd`	Deterministic limits on work and spend

`--allowedTools` isn't a boundary. Bash rules match command text (AI09 L3). The trailing `*` matters: per the docs, `Bash(git diff*)` would also match `git diff-index`.

Independent review

TS 4.6 says that a session that generated code is less effective at reviewing it, as it retains its reasoning. So the reviewer is a separate instance.

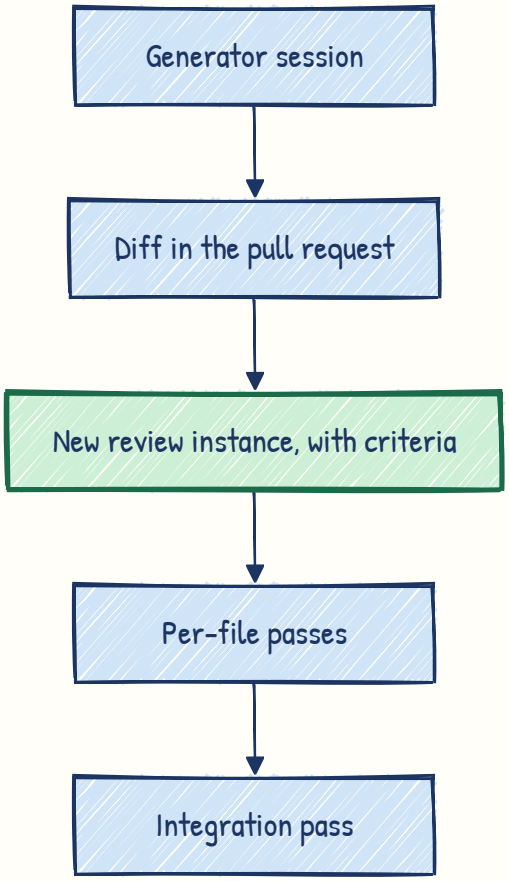


Figure 2. The reviewer sees the diff and criteria, and doesn't see how the code came to be. That independence is what you're paying for.

The guide adds multi-pass review: per-file passes for local issues, and an integration pass for cross-file data flow, which avoids attention dilution and contradictory findings. The best practices page makes the independence point simply: "A second session is also the easiest independent reviewer you can get."

Findings across commits

A re-run after a new commit posts the same comments, unless it knows about earlier ones. TS 3.6's answer: include prior findings, and report only what's new or unaddressed. My wrapper checks in code as well:

```
first run: exit 1, 2 new, blocking
after a commit, no memory of prior findings: 2 new, passing [log-pii, no-timeout]
after a commit, with prior findings: exit 0, 1 new, passing [no-timeout]
high finding accepted by a person: exit 0, 1 new, passing
```

The second line shows the duplicate: `log-pii` moved from line 90 to line 52. A key built on line numbers would treat it as new, so **fingerprint by file, rule and code, and leave out the line number.** Accepted findings also need recording, or they return each time.

Test generation

Problem	Fix
Duplicates of existing tests	Provide existing test files in context
Low-value tests	Put criteria and fixtures in CLAUDE.md
Weak coverage reasoning	Few-shot examples (TS 4.2)
Tests that only mirror the code	Have tests reviewed independently

Blocking or not?

TS 4.5 again: a pre-merge check is synchronous. A nightly audit can use Batch, through the API, since it's cheaper and has no latency SLA.

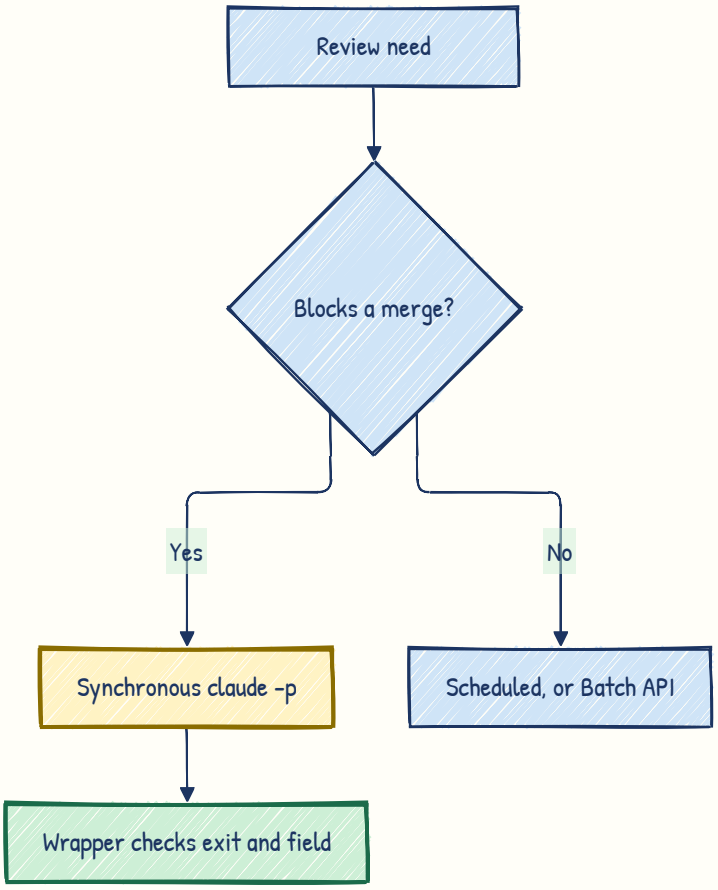


Figure 3. If someone's waiting, it must be synchronous. A failed run has to be visible, and never read as a pass.

GitHub Actions

`anthropics/claude-code-action@v1`, from the docs. Quick setup is `/install-github-app`. There are two modes:

Mode	Trigger	Results
Interactive, no `prompt`	`@claude` in a comment	Comment on the issue or PR
Automation, with `prompt`	Any GitHub event	Workflow log, unless told to post

Inputs include `prompt`, `claude_args`, `settings`, `anthropic_api_key`, and more. Secrets stay in GitHub Secrets. On public repositories, GitHub withholds secrets from runs triggered by fork pull requests. Triggering users need write access, unless you allow others. Also available is a hosted **Code Review** product, and GitLab has its own page. Both are outside the guide. **I haven't run the action.**

CI security

A pipeline reads files that a contributor wrote. So treat it as an agent reading untrusted content (AI09 L2).

Risk	Control
A malicious PR edits CLAUDE.md, or code comments, to steer review	Read config from the base branch where you can. Don't give the job write tools
`-p` runs project hooks and `.mcp.json` in an untrusted folder	Per the docs, there's no trust dialog with `-p`, so use `--bare`, or don't check out fork code with secrets
Secrets in the runner	Scoped tokens, and no secrets on fork PRs
A wide `Bash` allowance	Allow narrow commands. Use sandboxing or an isolated runner
Runaway cost	`--max-turns`, `--max-budget-usd`, and a workflow timeout
Claude's approval as a gate	Keep people or branch protection as the last word

The second row is the sharp one. From the docs: "A `-p` session shows no workspace trust dialog and no per-server approval prompt." A repository's `.claude/settings.json` hooks would run. In a job holding secrets, that's code execution by whoever wrote the PR.

Why the obvious fixes fall short

Obvious idea	Why it falls short
Set `CLAUDE_HEADLESS`, or use `--batch`	Neither is real
Parse text with a regex	Breaks, and hides failures
Same session reviews its own code	Retains its reasoning
Delete old comments before each run	Loses history, and doesn't stop repeats
Treat an empty result as a pass	Can't tell it from a failed run
Run on fork PRs with secrets	Hostile input with credentials
Move a blocking check to Batch	No latency guarantee

How Anthropic solved it

Mechanism	What's published	Source
Headless runs	`-p`, output formats, schemas	Headless docs
Deterministic limits	`--max-turns`, `--max-budget-usd`	Best practices, CLI docs
Bare mode	Same result on each machine	Headless docs
GitHub Action	Interactive and automation modes	GitHub Actions docs
Trust caveat	No dialog under `-p`	Headless and permissions docs
Independent review	Second-session reviewer	Best practices, exam guide

The production architecture

This is a recommended reference design, not a specific company's system.

Baseline: a job that runs Claude and prints text.

Improved: `-p` with JSON and inline comments. New problem: repeats pile up, a failed run looks like a pass, and a hostile PR can steer it.

Production: a wrapper with fail-closed logic, memory of findings, and limited trust.

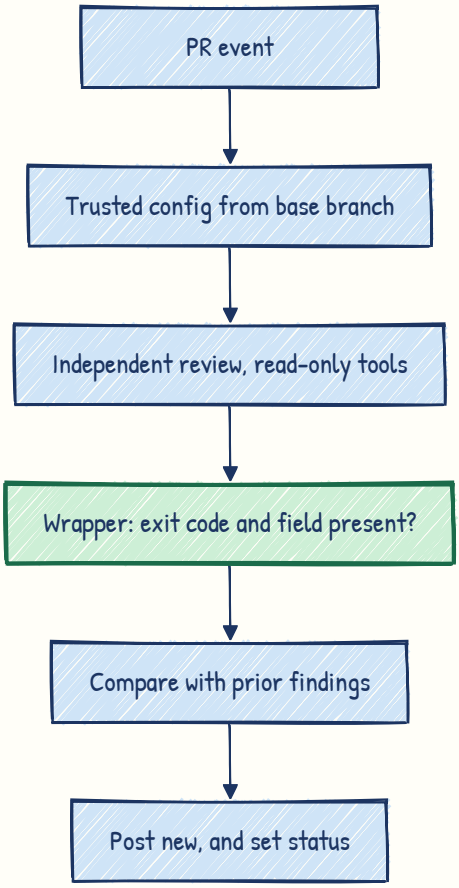


Figure 4. The wrapper owns the decisions. The model produces findings, and code decides what they mean for the merge.

Component	Why it exists	How it communicates
Trigger rules	No secrets on untrusted code	CI config
Trusted config	Stops a PR rewriting its own review	Base branch checkout
Review run	Findings, with no write tools	`claude -p`
Schema	Machine-readable findings	`--json-schema`
Wrapper	Pass, fail, or "didn't run"	Exit codes
Findings store	Dedupes, and records acceptances	PR comments or a database
Limits	Cost and time	Flags and timeout
Evals	Precision by category (TS 4.1)	Sampled later (AI10 L3)

Failures:

Failure	How you'd notice	What to do
Hangs	Timeout	Use `-p`
Missing `structured_output`	Wrapper check	Fail visibly, and retry once
Tool denied silently	Weak findings, `permission_denials`	Pre-approve what it needs
Duplicate comments	Developers complain	Fingerprints and prior findings
High false positives	Dismissal rate	Explicit criteria, `detected_pattern` (TS 4.4)
Injection through the diff	Odd findings, or actions	Read-only tools, and treat as untrusted
MCP server skipped	`mcp_server_errors` in stream-json	Fail CI on it, per the docs
Cost spike	Budget alerts	Limits and concurrency controls

Java

The two checks that make a job honest come before reading any findings:

```
if (!json.contains("\"structured_output\"")) {
```

The fingerprint leaves out the line number, and normalizes whitespace, so a moved issue still matches:

```
return file + "|" + rule + "|" + snippet.replaceAll("\\s+", " ").trim();
```

A bug of mine worth knowing. My first regex for findings didn't allow escaped quotes in a snippet, so it silently dropped a high-severity finding containing `\"`, and the job passed. The lab's output said "1 new, passing," and I caught that by reading it. **Use a real JSON library.** A parser that drops what it doesn't understand is another way of failing open.

Trade-offs

Decision	Benefit	Cost
Blocking review	Stops issues pre-merge	Latency, and false positives block people
Advisory comments	Low friction	Can be ignored
`--bare`	Reproducible	Must pass context explicitly
Project discovery	Simple	Differs by machine, and runs repo config
Multi-pass	Better depth	More tokens
Strict fail-closed	Honest	Provider outages block merges, so decide on an override

When not to block merges: while false positives are high. Run it as advisory, measure precision by category, and then promote categories that have earned it.

Common mistakes

Reading absence as success. A missing field, a non-zero exit or a skipped server must never produce a green check.

Common mistakes

Running with secrets on untrusted forks. Project hooks and MCP servers can run under `-p` with no prompt. Isolate those runs.

What to say in an interview

"I run `-p` with JSON output and a schema, in an instance independent of whatever generated the code, with CLAUDE.md or explicit flags for context. The wrapper checks the exit code and that `structured_output` exists, so a failed run can't look like a clean one. It dedupes by a fingerprint without line numbers. Tools are read-only, secrets don't reach fork PRs, and a person or branch protection keeps the last word."

Certification lens

How this lesson maps to CCAR-F

TS 3.6 and TS 4.6 are exam content: `-p`, `--output-format json`, `--json-schema`, CLAUDE.md for CI context, independent instances, prior findings, existing tests in context, and per-file plus integration passes. `--bare`, `dontAsk` and the Action's details aren't in the guide.

These are original practice questions, not real or leaked.

Q Q1 · Exam style, scenario: CI/CD · Easy. A script needs Claude's review output as data it can post as PR comments. Which flags fit?

A. `--output-format json` and `--json-schema`

B. `--batch` and `--parse`

C. `--verbose` and a log scraper

D. `--ci` and `--comments`

A **A** (TS 3.6). They give schema-shaped JSON. B and D aren't real, and C scrapes unstructured logs.

Q Q2 · Exam style, scenario: CI/CD · Medium. Claude generated a change and then, in the same session, approved it. A bug reached production. What's the best process change?

A. Ask the session to think harder when reviewing

B. Add a self-review step at the end of generation

C. Require the review to run in an independent instance with no generation context

D. Lower the threshold for comments

A **C** (TS 3.6, 4.6). Independence removes the retained reasoning. A and B stay in the same context, and D doesn't address it.

Q Q3 · Exam style, scenario: CI/CD · Medium. Developers see the same comments after each new commit. What's the best fix?

A. Run the review once per PR, and ignore later commits

B. Include prior findings, and instruct Claude to report only new or unaddressed issues

C. Delete all prior comments before re-running

D. Randomize the review's output order

A **B** (TS 3.6). It uses context to avoid repeats. A misses new issues, C loses the history, and D changes nothing.

Q Q4 · Exam style, scenario: CI/CD · Medium. Generated tests keep repeating existing cases, and many have little value. What's the best pair of changes?

A. Raise the test count, and remove duplicates afterward

B. Use Batch, and a larger model

C. Ask for high confidence only, and a shorter output

D. Provide existing test files, and document valuable test criteria and fixtures in CLAUDE.md

A **D** (TS 3.6). One addresses duplicates, and the other defines value. The others don't provide that information.

Q Q5 · Your job reports success, and `structured_output` is absent. What should the wrapper do, and why?

A Fail visibly as "review didn't run." The docs say that the field can be missing, for example if Claude declined. Reading that as no findings gives a false green, which is worse than a red job. Log the result, retry once if appropriate, and alert if it persists.

Revision - Lesson 5

→ `-p` runs non-interactively. `--batch`, `--ci` and `CLAUDE_HEADLESS` aren't real.

→ `--output-format json` with `--json-schema` gives `structured_output`.

→ Check exit code and the field, since a failed run must not look clean.

→ CLAUDE.md provides CI context (guide). `--bare` skips it, and is recommended for scripts (current docs).

→ Unattended runs deny what needs approval. Pre-approve narrowly. Limit turns and budget.

→ Review with an independent instance. Per-file passes, then integration.

→ Include prior findings. Fingerprint without line numbers, and record acceptances.

→ Provide existing tests, and document criteria and fixtures.

→ Blocking checks are synchronous. Non-blocking work can use Batch.

→ `-p` shows no trust dialog, so don't run fork code with secrets. Treat diffs as untrusted.

Your notes

Module Revision

Self-assessment

#	Can you...
1	Describe Claude Code as an agent loop?
2	Separate deterministic from non-deterministic instructions?
3	Choose among Grep, Glob, Read, Write, Edit?
4	Explore a codebase incrementally?
5	Trace a function through re-exports?
6	Give Edit's three checks, and the fallback?
7	Name the permission modes?
8	Place an instruction at the right scope?
9	Say how CLAUDE.md files load and combine?
10	Use imports, and say what they don't save?
11	Write a path rule, and name how it fails silently?
12	Choose between glob rules and directory files?
13	Say what belongs in CLAUDE.md?
14	Diagnose a rule that isn't followed?
15	Choose among CLAUDE.md, skill, subagent, hook?
16	Explain commands and skills, and how the docs merged them?
17	Use `context: fork` and `argument-hint`?
18	Give both readings of `allowed-tools`?
19	Define a subagent, with real tool limits?
20	Explain the Task and Agent naming?
21	State hook exit codes, and how a hook fails open?

#	Can you...
22	Explain why an Edit hook can miss a file?
23	Place MCP servers by scope, with secrets?
24	Choose between plan mode and direct execution?
25	Use Explore?
26	Choose among examples, tests and the interview pattern?
27	Decide whether issues go together?
28	Explain why a pass-or-fail check matters?
29	Use resume, fork, clear and compact correctly?
30	Run Claude Code in CI, with structured output?
31	Explain why "no findings" and "no review" must differ?
32	Explain `--bare`, and how it affects CLAUDE.md?
33	Set up independent, multi-pass review?
34	Dedupe findings across commits?
35	List the CI security risks?

Where this module is used

Idea	Where it returns
All of Domain 3	AI13
Hooks and gates on the exam	AI13, with AI06 L8
Sessions and context	AI13, with AI06 L7

Working space